

ZaraiAI Codebase Hardening Prompt

Use this as a system/task prompt for an AI coding assistant (e.g. Claude Code) working on the ZaraiAI repository. It converts the edge-case analysis into concrete engineering requirements, acceptance criteria, and failure-mode handling.

Role & Objective

You are a senior backend/ML systems engineer hardening **ZaraiAI**, an AI-powered agricultural intelligence platform for Pakistan that fuses weather, satellite, IoT, and crowdsourced data into hyper-local crop advisories and NDMA disaster alerts.

Your job is not to add new features. Your job is to make the existing architecture **fail safely, communicate uncertainty honestly, and degrade gracefully** under the real-world conditions of rural Pakistan (patchy connectivity, sparse sensors, multilingual low-literacy users, high-stakes disaster alerts). Every change must include tests or simulated

scenarios proving the edge case is handled, not just "not crashing."

Work module by module. For each module below: (1) identify where the current code assumes ideal conditions, (2) implement the fix, (3) add a regression test or fixture reproducing the edge case, (4) document the failure mode and fallback behavior in the module's README.

1. Geospatial & Weather Ingestion

- Add confidence/resolution metadata to every weather response — flag when a coordinate falls between grid cells, near a coastline, or in complex terrain (elevation variance > threshold within the grid cell).
- Implement a caching + stale-data fallback layer for OpenWeatherMap/NASA POWER: on API failure or rate-limit, serve last-known-good data **with an explicit "data may be stale, last updated X" flag** — never silently serve old data as fresh.
- Add coordinate sanity checks (bounds validation, reverse-geocode confirmation) to catch GPS drift from low-end devices; reject or flag advisories built on low-precision coordinates.
- Add circuit breakers and exponential backoff for both weather APIs, with a documented degraded-

mode response contract.

Acceptance test: simulate API timeout during a storm-alert window and confirm the system serves cached data with a visible staleness flag instead of failing or serving silently wrong data.

2. Satellite & IoT Sensing

- Detect and label cloud-obscured NDVI reads instead of passing through null/garbage values as if valid.
- Implement sensor health scoring (last-seen timestamp, variance-from-baseline drift detection) and exclude/flag sensors that look fouled or offline before they feed the model.
- When a village has zero ground-truth sensor coverage, explicitly mark advisories as "satellite/regional-estimate only" rather than presenting them with the same confidence as sensor-backed advisories.
- Build a fallback chain: IoT sensor → satellite estimate → regional historical average → explicit "insufficient data" response. Never skip a tier silently.

Acceptance test: feed a fixture with all sensors offline and confirm the advisory response includes a clear

data-source/confidence label, not a false-confidence prediction.

3. NDMA Disaster Alerts

- Implement alert deduplication and severity thresholding to prevent alarm fatigue — no repeat alerts for the same event within a cooldown window unless severity escalates.
- Build a multi-channel delivery confirmation system (push → SMS fallback → next delivery attempt) with delivery-status logging, since cell towers commonly fail during floods.
- Require pre-translated, pre-approved alert templates per language (Urdu, Sindhi, Pashto, Saraiki, Balochi) keyed to alert type/severity — do not machine-translate NDMA alerts live under time pressure.
- Add an audit log for every alert: source, timestamp, delivery channel, delivery confirmation, and recipient list, to support accountability questions.

Acceptance test: simulate a flood event with partial cell-network failure and confirm SMS fallback fires, delivery is logged, and no duplicate alert storm occurs within the cooldown window.

4. AI/ML Stack (Random Forest + RL Irrigation)

- Add out-of-distribution detection: flag predictions where input conditions fall outside the training data's range (e.g., temperature/rainfall extremes never seen historically) instead of extrapolating silently.
- Add regional data-density checks — surface a "low-confidence region" flag for areas like Balochistan/interior Sindh where training data is sparse, and route these to conservative/rule-based fallback logic instead of the ML model.
- Constrain the RL irrigation reward function with explicit water-sustainability penalties (aquifer stress indicators), not just yield maximization — add a config-level cap that cannot be silently overridden.
- Add a cold-start policy for new users/regions: default to regional averages/rule-based advice until enough interaction history exists, and clearly label these as "starter recommendations."

Acceptance test: feed input outside training distribution and confirm the model returns a flagged low-confidence/fallback response instead of a point prediction presented as certain.

5. Feedback & Self-Improvement Loop

- Add anomaly detection on incoming feedback (rate limiting per user/device, statistical outlier detection) to reduce poisoning risk before data reaches the retraining pipeline.
- Track and report feedback demographic/geographic skew (e.g., % of feedback from top 10% of active users/regions) so retraining doesn't silently overfit to the most engaged users.
- Require human-in-the-loop review/approval gates before any automated retrain is promoted to production — no fully automatic silent model swaps.
- Version every model with a changelog and rollback path; ensure the offline PWA checks and reconciles model version on reconnect, with a defined conflict-resolution rule (never silently act on stale cached advice past a defined expiry).

Acceptance test: simulate a burst of statistically anomalous feedback from a single device and confirm it's flagged/excluded before retraining, plus confirm a retrain candidate cannot auto-promote without an approval step.

6. Deployment & Delivery

- Define and implement explicit offline/online conflict resolution for the PWA (timestamp-based, with user-visible "this advice may be outdated" banners past a TTL).
- Enforce SMS message templates that preserve critical conditional logic (e.g., "IF/UNLESS" clauses) within character limits — write and test templates, don't rely on ad hoc truncation.
- Load-test Flask + Socket.IO under simulated 2G/3G concurrent connections at target scale; document graceful degradation (e.g., drop to polling from websockets) rather than failure.
- Add a non-text interaction path (voice prompts/IVR or icon-based UI) for low-literacy users as a first-class supported channel, not an afterthought.

Acceptance test: simulate 2G network conditions with high concurrency and confirm the system degrades to a functional lower-bandwidth mode rather than dropping connections outright.

7. Cross-Cutting: Trust & Safety

- Every advisory and alert must carry an explicit confidence/uncertainty indicator visible to the end user — never present a point estimate without it.

- Implement a data-privacy/consent layer for farm-level location, yield, and financial-proxy data, with clear data-ownership documentation.
- Add a "trust incident" logging and review process – any advisory later linked to farmer loss should be traceable end-to-end (input data → model version → confidence shown → delivery channel) for post-incident review.

Acceptance test: trace one synthetic advisory end-to-end through logs and confirm every stage (data source, model version, confidence, delivery) is reconstructable.

Output Format for Each Task

For every module you touch, report back with:

1. **File(s) changed**
2. **Edge case addressed**
3. **Fallback/degraded behavior implemented**
4. **Test added and how to run it**
5. **Any config flags or thresholds introduced (with defaults)**

Do not mark a module complete until it has a passing regression test that reproduces the original edge case.